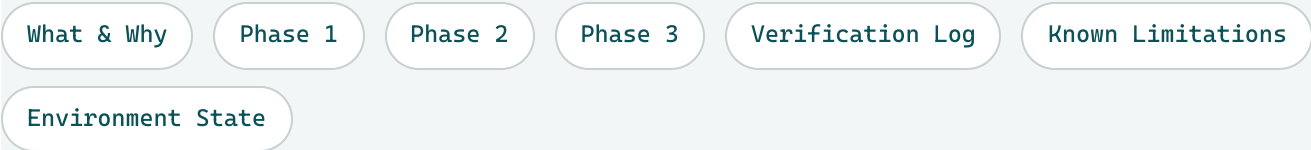


Raw-to-Queryable Data Pipeline

A standalone, deep-dive reference for Build 2 — every phase, every file, every verification result, in full. Written retroactively, per standing procedure, alongside the same standard as every other build's brief in this project. Complements (does not replace) the living cheat sheet and the phase diagrams.

STATUS	COMMITTS	CONTINUES
Complete — all 3 phases	f5468e0 → 68b810c	Build 1 (Ask-Your-Database agent)
PRECEDES		
Build 2.5 (agent scratch workspace)		



What Was Built, and Why

Build 1 gave the agent a database to query, but that database was hand-seeded toy data (four customers, five orders). Build 2 gives it something real: a genuine equipment-checkout export, landed unmodified, then deliberately transformed in a separate, later step — the classic ELT split, done as two distinct schemas (**raw** then **clean**) rather than one script that ingests and cleans in the same breath.

The two-schema split is the core design decision: **raw.allocations** is all-TEXT, no constraints, mirroring the source file exactly — its entire job is preserving what the source actually said before any decision gets made about what it means. **clean.allocations** is where those decisions happen: real types, a generated column, deliberate NULL handling. Keeping them as two tables (not one, cleaned in place) means the original import is always available to re-derive from if a transformation decision turns out wrong later.

1 The Raw Ingestion Layer

Goal: land a genuinely messy real-world CSV completely unmodified in a staging table, and verify the row count matches the source exactly. No transformation yet — that's deliberately a later phase.

The source: a real equipment-checkout export (ALLOCATION-export-Oct_2024-2025.csv, 1,535 rows) from an actual institution, kept outside this repo. Its Patron Email column was already exported as domain-only (e.g. @artschool.edu) rather than full addresses — a privacy choice made at export time, not something this project added. The real institution's domain was later masked to the fictional artschool.edu before this became a public-facing portfolio piece (see project_agent-roadmap.md memory for that masking pass in full).

```
004_raw_schema.sql (originally raw_schema.sql, renamed when SQL files were numbered)
```

```
CREATE SCHEMA IF NOT EXISTS raw;
```

```
DROP TABLE IF EXISTS raw.allocations;
```

```
CREATE TABLE raw.allocations (  
  allocation_id      TEXT,  
  patron_department TEXT,  
  patron_email       TEXT,  
  renewal_count      TEXT,  
  actual_start       TEXT,  
  scheduled_end      TEXT,  
  actual_end         TEXT,  
  duration           TEXT,  
  resource_count     TEXT,  
  summary            TEXT  
);
```

```
GRANT USAGE ON SCHEMA raw TO appdb_reader;
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA raw TO appdb_reader;
```

```
ALTER DEFAULT PRIVILEGES IN SCHEMA raw GRANT SELECT ON TABLES TO appdb_reader;
```

Every column is TEXT, on purpose. Nothing here is cleaned or coerced — not even obviously numeric fields like `renewal_count`. Deciding what a value *means* (a number? a timestamp? a real NULL?) is explicitly Phase 2's job, not this one's. Mixing the two would make it impossible to tell, later, whether a "clean" value reflects the source or an ingestion-time assumption.

```
ingest.py
```

```

import csv
import os
from dotenv import load_dotenv
import psycopg

load_dotenv(encoding="utf-8-sig")

CSV_PATH = r"C:\IMAGEBANK_2023\DOCKER_SQL\EMEC_AGENT_BUILD\ALLOCATION-export-Oct_2024-2025.csv"

conn = psycopg.connect(
    host=os.environ.get("POSTGRES_HOST", "127.0.0.1"),
    port=5432,
    dbname=os.environ["POSTGRES_DB"],
    user=os.environ["POSTGRES_USER"],
    password=os.environ["POSTGRES_PASSWORD"],
)

with open(CSV_PATH, newline="", encoding="utf-8-sig") as f:
    reader = csv.DictReader(f)
    rows = [
        (row["Allocation"], row["Patron Department"], row["Patron Email"],
         row["Renewal Count"], row["Actual Start"], row["Scheduled End"],
         row["Actual End"], row["Duration"], row["Resource Count"], row["Summary"])
        for row in reader
    ]

with conn.cursor() as cur:
    cur.executemany(
        """INSERT INTO raw.allocations (
            allocation_id, patron_department, patron_email, renewal_count,
            actual_start, scheduled_end, actual_end, duration,
            resource_count, summary
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)""",
        rows,
    )
conn.commit()
conn.close()
print(f"Loaded {len(rows)} rows into raw.allocations")

```

Why csv.DictReader, not pandas, for ingestion specifically: csv.DictReader guarantees every field arrives as a plain string, with nothing silently type-coerced. Pandas' automatic type inference is exactly the kind of decision this layer is designed to defer to Phase 2. Pandas still gets used — one phase later, for profiling.

Verified live: `SELECT count(*) FROM raw.allocations;` → 1535, matching the source CSV's row count exactly.

2 Profile & Transform

Goal: understand the raw data's actual messiness before deciding the clean schema, then write the transformation that turns text into real types — deliberately, not by pandas' automatic inference.

```
profile_raw_data.py (named profile.py at the time – renamed during Build 3 after it was found
silently shadowing Python's stdlib profile module)

print("--- patron_department: blank rate + distinct values (repr shows hidden whitespace) ---")
print("Blank count:", (df["patron_department"] == "").sum())
print(sorted(repr(v) for v in df["patron_department"].unique()))

print("--- date format consistency: actual_start / scheduled_end / actual_end ---")
for col in ["actual_start", "scheduled_end", "actual_end"]:
    parsed = pd.to_datetime(df[col], format="%m/%d/%Y %I:%M %p", errors="coerce")
    print(f"{col}: {parsed.isna().sum()} rows failed to parse")

print("--- duration sanity check ---")
computed_seconds = (ends - starts).dt.total_seconds()
mismatch = computed_seconds.astype("Int64") != df["duration"].astype("int64")
print("Rows where computed != stored duration:", mismatch.sum())
```

What profiling actually found, and why each finding mattered: patron_department had real blank strings mixed with real values — not garbage, a genuine "not applicable" case that needed to become a true NULL, not stay as ''. All three date columns parsed cleanly against one exact format (MM/DD/YYYY H:MM AM/PM) with zero failures. The stored duration column's values matched a freshly computed actual_end - actual_start exactly — meaning clean.allocations could *compute* duration instead of trusting a redundant stored field, closing off an entire class of future drift between the two.

005_clean_schema.sql

```
CREATE SCHEMA IF NOT EXISTS clean;
```

```
DROP TABLE IF EXISTS clean.allocations;
```

```
CREATE TABLE clean.allocations (
    allocation_id      TEXT PRIMARY KEY,
    patron_department  TEXT,
    patron_email_domain TEXT NOT NULL,
    renewal_count      INTEGER NOT NULL,
    actual_start       TIMESTAMP NOT NULL,
    scheduled_end      TIMESTAMP NOT NULL,
    actual_end         TIMESTAMP NOT NULL,
    duration_seconds   INTEGER GENERATED ALWAYS AS
        (EXTRACT(EPOCH FROM (actual_end - actual_start)))::INTEGER) STORED,
```

```

resource_count          INTEGER NOT NULL,
summary                 TEXT
);

GRANT USAGE ON SCHEMA clean TO appdb_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA clean TO appdb_reader;
ALTER DEFAULT PRIVILEGES IN SCHEMA clean GRANT SELECT ON TABLES TO appdb_reader;

```

duration_seconds is **GENERATED ALWAYS AS (...) STORED, not a plain inserted column**. Postgres computes and locks it from `actual_start/actual_end` in the same row — structurally impossible for it to drift out of sync, unlike the raw layer's separately-stored duration field that profiling proved (for this dataset) happened to already agree.

transform.py

```
DATE_FORMAT = "%m/%d/%Y %I:%M %p"
```

```

with conn.cursor() as cur:
    cur.execute("SELECT * FROM raw.allocations")
    columns = [desc[0] for desc in cur.description]
    raw_rows = cur.fetchall()

rows = []
for row in raw_rows:
    record = dict(zip(columns, row))
    rows.append((
        record["allocation_id"],
        record["patron_department"] or None,
        record["patron_email"],
        int(record["renewal_count"]),
        datetime.strptime(record["actual_start"], DATE_FORMAT),
        datetime.strptime(record["scheduled_end"], DATE_FORMAT),
        datetime.strptime(record["actual_end"], DATE_FORMAT),
        int(record["resource_count"]),
        record["summary"],
    ))

```

```

with conn.cursor() as cur:
    cur.executemany(
        """INSERT INTO clean.allocations (
            allocation_id, patron_department, patron_email_domain, renewal_count,
            actual_start, scheduled_end, actual_end,
            resource_count, summary
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)""",
    )

```

```
        rows,  
    )  
    conn.commit()
```

record["patron_department"] or None is the one deliberate cleaning decision in this whole script. Python treats an empty string as falsy, so a blank department becomes a real SQL NULL instead of staying an empty string — the exact gap profiling surfaced. Every other field is a direct type conversion (`int()`, `datetime.strptime()`), not a judgment call.

Verified live: `SELECT count(*) FROM clean.allocations;` → 1535, matching `raw.allocations` exactly.
`patron_email_domain` distinct values: `@inside.artschool.edu` (946), `@artschool.edu` (571), `@alumni.artschool.edu` (9), `@gmail.com` (9) — real categorical signal, confirming the domain-only export wasn't lossy for this dataset's purposes.

Also delivered this phase: the SQL files were numbered (001–005) to encode their run order explicitly, and `sql-test-01-cheatsheet.html` — the command-reference living document every later build has appended to since — was started.

3 Agent Integration & Data Quality

Goal: let the agent actually see `clean.allocations`, and codify what "correct" means for that table as real, automated tests — the first automated test suite in this project.

```
tools.py - schema discovery widened  
  
WHERE table_schema IN ('public', 'clean')
```

Deliberately not raw. `appdb_reader`'s grants extend across `public`, `raw`, and `clean`, but `tools.py`'s schema-discovery query only surfaces `public` and `clean` to the agent. `raw` stays internal-only — a real distinction between what a role is *permitted* to touch and what the agent is actually told exists.

Real gap found and fixed mid-phase: the first cross-table question exposed a real self-correction moment — the model tried an unqualified table name, got a genuine Postgres error back, and retried with the `clean.` prefix on its own without being told to. Not a bug, but the first live proof that graceful error recovery (not just error *handling*) was actually working end to end.

```
test_data_quality.py  
  
import pandera.pandas as pa  
  
CLEAN_SCHEMA = pa.DataFrameSchema({  
    "allocation_id": pa.Column(str, unique=True, nullable=False),  
    "patron_department": pa.Column(str, nullable=True),  
    "patron_email_domain": pa.Column(str, nullable=False),  
    "renewal_count": pa.Column(int, pa.Check.ge(0), nullable=False),  
    "resource_count": pa.Column(int, pa.Check.ge(0), nullable=False),  
    "duration_seconds": pa.Column(int, pa.Check.ge(0), nullable=False),  
})  
  
def test_raw_row_count():  
    assert len(_query("SELECT * FROM raw.allocations")) == 1535  
  
def test_clean_row_count_matches_raw():  
    raw = _query("SELECT COUNT(*) AS n FROM raw.allocations")  
    clean = _query("SELECT COUNT(*) AS n FROM clean.allocations")  
    assert raw["n"][0] == clean["n"][0]  
  
def test_clean_department_has_real_nulls():  
    clean = _query("SELECT patron_department FROM clean.allocations")  
    assert clean["patron_department"].isna().sum() > 0  
  
def test_clean_schema_types_and_constraints():
```



```
clean = _query("SELECT allocation_id, patron_department, patron_email_domain, "  
              "renewal_count, resource_count, duration_seconds FROM clean.allocations")  
CLEAN_SCHEMA.validate(clean)
```

Why pandera, not scattered asserts: DataFrameSchema declares an entire table's expected shape — types, nullability, value constraints — as one piece of data rather than a pile of ad hoc checks.

test_clean_department_has_real_nulls is the direct regression guard for Phase 2's blank→NULL decision: if a future change ever silently reverted to storing empty strings, this test would fail loudly instead of the gap going unnoticed.

Result: all 4 tests passing on first run, connecting as appdb_reader (not the owner role) — the same read-only credential the agent itself uses.

Verification Log

What was actually exercised against the running system, not just read for plausibility — same standard as every build in this project.

Raw row count

`raw.allocations` — 1,535 rows, matching the source CSV exactly.

Clean row count

`clean.allocations` — 1,535 rows, matching `raw.allocations` exactly; no rows silently dropped during transform.

Profiling findings, each acted on

Blank `patron_department` values confirmed real (→ NULL handling); all three date columns parsed cleanly against one exact format; stored `duration` matched a freshly computed value exactly (→ became a **GENERATED** column instead of a trusted stored one).

Domain distribution, post-transform

`@inside.artschool.edu` (946), `@artschool.edu` (571), `@alumni.artschool.edu` (9), `@gmail.com` (9) — real categorical signal preserved through the transform.

Agent self-correction, live

First cross-schema question: model tried an unqualified table name, got a real Postgres error, retried with the `clean.` prefix unprompted, then answered correctly.

Automated test suite

4/4 `test_data_quality.py` tests passing, run as `appdb_reader` — the first automated tests in this project, replacing what had been entirely manual verification through Build 1.

Known Limitations & Open Items

list_tables/describe_table as two separate calls, not one. The first real cross-table question triggered 3 separate describe_table calls in one turn — the model exploring every discovered table, not just the one the question needed. Not a bug (the agent still answered correctly, token cost negligible on Haiku-tier), but a genuine, low-priority efficiency idea: merging the two tools into one "describe everything" call would cut round-trips without changing how much schema information the model receives. Flagged for later, not fixed now — carried forward into Build 2.5's brief, still unaddressed as of that build too.

raw excluded from agent-visible schema discovery, by design. appdb_reader can technically SELECT from raw.allocations (the grant is there), but tools.py's discovery query never surfaces it to the model. Intentional, not a gap — worth remembering if a future build ever needs the agent to reason about data provenance, since today it structurally cannot see the raw layer exists.

Resolved during this build — no automated tests, at all, through Build 1. Every Build 1 verification was manual and interactive. Build 2 Phase 3's test_data_quality.py is the first automated suite in the project, establishing the pandera + pytest pattern every later build's own test suite has since followed.

Environment State

COMPONENT	DETAIL
New schemas	raw, clean
New SQL files	004_raw_schema.sql, 005_clean_schema.sql (numbered from raw_schema.sql this phase)
New/changed Python	ingest.py, profile.py (later renamed profile_raw_data.py), transform.py, tools.py (widened discovery), test_data_quality.py
New dependency	pandas, pandera
Real source data	ALLOCATION-export-Oct_2024-2025.csv, 1,535 rows, outside this repo
Living doc started	sql-test-01-cheatsheet.html
Role used	appdb_reader (grants extended to raw + clean; no new role needed for this build)

Build 2 complete. Next (chronologically): Build 2.5 — agent scratch workspace. This brief was written retroactively per standing procedure, reconstructed from git history (f5468e0→68b810c), the actual current files, and project_agent-roadmap.md memory — not from a live session transcript, unlike Builds 2.5/3's briefs. Ongoing cross-build command reference lives in sql-test-01-cheatsheet.html.